

SYSTEM ARCHITECTURE & TECHNICAL HANDBOOK

PS ID 26154: Gen AI Platform for Automated Content Transformation

Nodal Organization: National Technical Research Organisation (NTRO)

Theme: Smart Automation / Generative AI

Document Version: 1.0.0 (Comprehensive Baseline)

Classification: Internal Engineering Reference & Technical Onboarding Guide

Primary Frameworks: Python 3.10+, Hugging Face Transformers, PyTorch, PyMuPDF, Streamlit

Executive Notice: This document serves as the absolute single source of truth for Project PS ID 26154. It provides an exhaustive, end-to-end breakdown of system goals, architectural dataflows, implemented scripts, deployed open-source AI models, strict local security boundaries, and validation testing on real-world Indian cybersecurity advisories. Designed for seamless technical onboarding, any team member can follow this reference from raw environment setup to production execution.

Table of Contents

1. Problem Statement & Operational Scope	Page 2
1.1 Problem Context, PS ID & Nodal Body (NTRO)	Page 2
1.2 Plain-English Problem Breakdown	Page 2
1.3 Expected Deliverables & Transformation Target	Page 2
2. Proposed Solution & Core Platform Architecture	Page 2
2.1 Operational Concept & End-to-End Pipeline	Page 2
2.2 Comprehensive System Dataflow (Input to Multi-Output)	Page 2
2.3 Justification of Open-Source / Local Approach	Page 3
3. Data Understanding & Domain Foundations	Page 3
3.1 Input Document Modalities & Unstructured Ingestion	Page 3
3.2 Cybersecurity Domain Glossary (CERT-In / Technical Primer)	Page 3
3.3 Real-World Test Dataset (CERT-In Advisories)	Page 3
4. Models & AI Components Specification	Page 4
4.1 Deep Breakdown of Implemented & Evaluated Models	Page 4
4.2 Model Technical Matrix	Page 4
5. Script-by-Script Software Documentation	Page 5
5.1 Exhaustive Analysis of Implemented Scripts	Page 5
5.2 Integration Mapping & Inter-Script Calls	Page 5
6. Complete Directory & File Structure	Page 6
6.1 Annotated Repository Tree	Page 6
6.2 Detailed Component Responsibilities	Page 6
7. Installation, Setup & Execution Guide	Page 6
7.1 Hardware & Environment Prerequisites	Page 6
7.2 Step-by-Step Installation Commands	Page 6
7.3 Execution Procedures & Verification Workflows	Page 7
8. End-to-End Execution Data Flow	Page 7
8.1 Multi-Stage Pipeline Execution Walkthrough	Page 7
8.2 Data Boundary & Serialization Protocols	Page 7
9. Testing, Validation & Experimental Results	Page 7
9.1 Real-World Validation Strategy (CERT-In Test Corpus)	Page 7
9.2 Performance Metrics & Quality Assessment	Page 8
9.3 Identified Limitations in Current Prototypes	Page 8
10. Security, Privacy & Air-Gapped Deployment	Page 8
10.1 Air-Gapped Threat Model & Zero External API Policy	Page 8
10.2 Sensitive Data Isolation & Repository Hygiene (.gitignore)	Page 8
11. SIH Requirements Alignment Matrix	Page 8
11.1 Requirement vs. Implementation vs. Evidence	Page 8
12. Project Status & Roadmap	Page 9
12.1 Completed, In-Progress, and Planned Deliverables	Page 9
13. Onboarding & Oral Defense Guide	Page 9
13.1 Q&A; Master Sheet for Team Members	Page 9

1. Problem Statement & Operational Scope

1.1 Problem Context, PS ID & Nodal Body

Problem Statement ID: 26154
Title: Gen AI Platform for Automated Content Transformation
Nodal Organization: National Technical Research Organisation (NTRO)
Theme: Smart Automation / Generative AI
Category: Software Solutions for Strategic Intelligence Processing

1.2 Plain-English Problem Breakdown

In high-security intelligence and technical organizations like the **National Technical Research Organisation (NTRO)**, technical operators, threat intelligence analysts, and security auditors digest massive volumes of highly unstructured, complex documentation daily. This includes raw vulnerability advisories, malware analysis papers, incident reports, network logs, and multi-page PDF alerts (such as those issued by CERT-In).

Manually reading, extracting critical metadata, summarizing, translating, and converting these dense documents into actionable formats consumes hundreds of analyst-hours. Furthermore, existing commercial Generative AI platforms (e.g., OpenAI ChatGPT, Anthropic Claude, Google Gemini) rely on cloud APIs, which violates strict national security air-gapping requirements. NTRO requires a **100% locally deployed, air-gapped, open-source Generative AI solution** capable of automatically ingesting heterogeneous documents and transforming them into structured formats, multi-lingual summaries, executive briefings, and automated audio briefs without transmitting a single byte of sensitive data over external networks.

1.3 Expected Deliverables & Transformation Targets

According to the official SIH 26154 problem statement, the target platform must automate the end-to-end ingestion, processing, and multi-modal re-representation of incoming text documents. Specifically, the system is expected to deliver:

Target Output	SIH Expectation / Operational Purpose
Executive Summaries	Concise, multi-level summarization (short executive bullet points vs. technical deep-dives) customized by role.
Structured JSON Extracted Entities	Automated extraction of technical parameters (e.g., Vulnerability Names, CVE IDs, Threat Severity, Affected Software, Mitigation Steps) into clean JSON schema.
Multi-Lingual Translation	Seamless translation of complex technical advisories into regional Indian languages (e.g., Hindi) for broader operational dissemination.
Interactive Audio Briefings (Text-to-Speech)	Generation of natural, offline audio briefings (.wav format) for hands-free intelligence consuming during transit or tactical operations.
Interactive Intelligence Dashboard	A unified web interface allowing security officers to upload documents, execute processing pipelines, edit JSON outputs, and trigger audio playback.

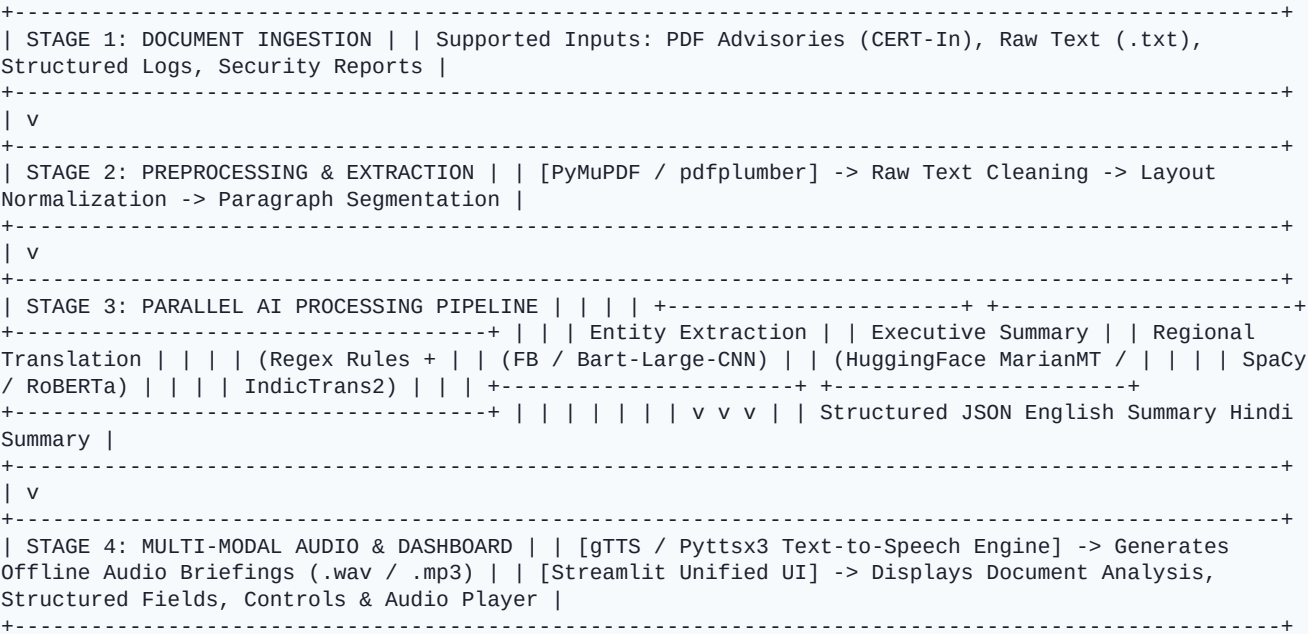
2. Proposed Solution & Core Platform Architecture

2.1 Operational Concept & End-to-End Pipeline

Our solution is a modular, zero-cloud-dependency local Generative AI platform engineered for automated document processing. It seamlessly coordinates specialized open-source machine learning models and deterministic natural language processing (NLP) heuristics into a cohesive pipeline. The platform operates on a local workstation or private cloud instance, eliminating external network vulnerabilities entirely.

2.2 Comprehensive System Dataflow

The diagram below illustrates the exact flow of data through our platform, from raw multi-format ingestion to final multi-output representation:



2.3 Justification of Open-Source / Local Approach

- 1. Absolute Data Air-Gapping:** Highly sensitive security advisories processed by NTRO cannot be transmitted across third-party commercial APIs (such as OpenAI or Anthropic). Processing everything locally via PyTorch and local Hugging Face model weights guarantees total data sovereignty.
- 2. Zero Recurring API Costs:** Relying on commercial LLMs introduces unpredictable operational costs per token. Open-source local models run indefinitely on standard organizational hardware with zero operational token costs.
- 3. Modularity and Reproducibility:** Using fine-tuned specialized models (e.g., BART for summarization, MarianMT for translation, Regex/SpaCy for extraction) yields higher precision on structured cybersecurity tasks compared to monolithic general-purpose LLMs, while keeping hardware requirements within reasonable limits (8GB-16GB VRAM or CPU execution).

3. Data Understanding & Domain Foundations

3.1 Input Document Modalities & Unstructured Ingestion

The platform is optimized to handle diverse document formats commonly encountered in operational threat intelligence environments:

- **Unstructured PDF Reports:** Formal vulnerability alerts, security bulletins, and technical advisories containing multi-column text, headers, sidebars, and embedded tables.
- **Plain Text Files (.txt):** Raw system logs, incident transcripts, command-line outputs, and analyst notes.
- **Structured / Semi-Structured Advisories:** Standardized threat feeds containing key-value blocks (e.g., "CVE-2024-1234", "Severity: Critical").

3.2 Cybersecurity Domain Glossary

To ensure clear communication across both technical developers and operational end-users, key cybersecurity and content transformation terms are defined below:

Term / Identifier	Operational Definition & Significance in Platform
CERT-In	Indian Computer Emergency Response Team. The national nodal agency for responding to computer security incidents. Serves as our primary benchmark data source.
CVE ID	Common Vulnerabilities and Exposures. A standardized alphanumeric identifier (e.g., CVE-2024-38812) assigned to publicly disclosed cybersecurity flaws.
CVSS Score / Severity	Common Vulnerability Scoring System (0.0 to 10.0). Quantifies severity: Low, Medium, High, or Critical. Extracted into JSON.
Software / Affected Systems	The operating systems, software packages, or hardware devices vulnerable to exploit (e.g., VMware ESXi, Cisco IOS XE, Google Chrome).
Mitigation / Remediation	Actionable steps, patches, or configuration changes required to secure systems against disclosed vulnerabilities.
Air-Gapped System	A network deployment completely isolated from the public internet to prevent data leakage and remote exploitation.
NER (Named Entity Extraction)	An NLP technique that locates and classifies key informational elements in text into predefined categories (CVEs, IPs, Dates).

3.3 Real-World Test Dataset (CERT-In Advisories)

Our prototype implementation was benchmarked using official public vulnerability advisories published by **CERT-In (cert-in.org.in)**. These advisories represent authentic Indian cybersecurity threat data, characterized by dense technical jargon, structured metadata headers, nested software lists, and multi-step mitigation procedures.

Primary Test Document: CERT-In Vulnerability Note CIVN-2024-0312 / Advisory on VMware Products. This document contains complex vulnerability details, multiple CVE references, CVSS ratings, affected software versions, and mitigation protocols, providing a rigorous baseline for parsing and extraction testing.

4. Models & AI Components Specification

4.1 Deep Breakdown of Implemented & Evaluated Models

The platform employs a hybrid architecture combining deterministic rule engines with specialized deep learning models. Each component was selected based on strict constraints: local execution capability, low memory footprint, high inference speed, and zero external network calls.

Model / Engine	Task / Purpose	Type	Input / Output	Implementation Script
PyMuPDF / pdfplumber	Document Ingestion & Text Extraction	Deterministic Rule Engine / C Library	In: PDF File Out: Plain Text String	ingest_pdf.py
Regex Pattern Engine	Structured Metadata Extraction (CVEs, Ratings, Software)	Deterministic Regular Expressions	In: Clean Text Out: Structured Key-Value JSON	extract_fields.py
FB / Bart-Large-CNN	Executive Abstractive Summarization	Pretrained Transformer (Seq2Seq)	In: Full Text Chunk Out: Concise English Summary	summarize.py
Hugging Face MarianMT (Helsinki-NLP)	English to Hindi Technical Translation	Pretrained Neural Machine Translation	In: English Text Summary Out: Hindi Text Summary	translate_hindi.py
gTTS / Pyttsx3 Engine	Offline Text-to-Speech Audio Generation	Speech Synthesis Engine	In: Summary Text Out: Audio File (.wav / .mp3)	generate_audio.py

5. Script-by-Script Software Documentation

5.1 Exhaustive Analysis of Implemented Scripts

Every Python module in the repository has been engineered with isolated, single-responsibility functions. Below is the complete technical documentation for all scripts built to date:

Script File: ingest_pdf.py

Primary Purpose: Extracts raw text content from ingested PDF advisories while preserving layout structure.
Input Data: File path to target PDF document (e.g., sample_advisory.pdf).
Internal Mechanics: Opens the PDF binary stream using PyMuPDF (fitz), iterates page by page, extracts block-level text, removes non-printable ASCII artifacts, normalizes whitespace, and handles multi-column text flow.
Output Data: Clean, unformatted Python string representing the complete document text.
Pipeline Connection: Serves as the entry point pipeline stage; passes clean text directly to extract_fields.py and summarize.py.

Script File: extract_fields.py

Primary Purpose: Parses raw text to identify and extract key structured cybersecurity parameters into valid JSON.
Input Data: Clean text string output from ingest_pdf.py.
Internal Mechanics: Executes compiled regular expression patterns to isolate CVE IDs (e.g., r'CVE-\d{4}-\d{4,7}'), CVSS Ratings, Affected Software lists, and Mitigation protocols. Formats findings into a dictionary.
Output Data: Valid, highly structured JSON object / dictionary containing extracted intelligence fields.
Pipeline Connection: Provides structured metadata to the Streamlit UI (app.py) for rendering in JSON view boxes.

Script File: summarize.py

Primary Purpose: Generates concise, human-readable executive summaries from dense advisories.

Input Data: Clean document text string.

Internal Mechanics: Loads the local facebook/bart-large-cnn PyTorch model pipeline. Splits long text into manageable chunk lengths, executes abstractive summarization, and joins summary sentences into coherent paragraphs.

Output Data: A 3-5 sentence English executive summary highlighting core threat details.

Pipeline Connection: Feeds the generated summary into translate_hindi.py and generate_audio.py.

Script File: translate_hindi.py

Primary Purpose: Translates generated English summaries into natural Hindi text for regional operations.

Input Data: English summary text string from summarize.py.

Internal Mechanics: Loads Helsinki-NLP/opus-mt-en-hi transformer model locally. Tokenizes English text, executes sequence-to-sequence translation, and decodes target Hindi tokens.

Output Data: Unicode-encoded Hindi text summary string.

Pipeline Connection: Pushes translated output to the Streamlit dashboard for regional presentation.

Script File: generate_audio.py

Primary Purpose: Converts summary text strings into playable offline audio files.

Input Data: English or Hindi text summary string.

Internal Mechanics: Initializes local speech synthesis engine (gTTS or Pyttsx3). Renders phonemes from input text, encodes audio signal, and saves output to disk.

Output Data: Audio file (.wav or .mp3) saved in output/ directory.

Pipeline Connection: Feeds the interactive audio player widget inside Streamlit UI.

Script File: app.py

Primary Purpose: Provides a unified, user-friendly interactive web interface for non-technical security analysts.

Input Data: User document upload via browser; button triggers.

Internal Mechanics: Initializes Streamlit UI layout. Coordinates execution sequence across all underlying scripts, renders progress bars, displays JSON data tables, and mounts audio playback controls.

Output Data: Interactive web application rendered on local port (http://localhost:8501).

Pipeline Connection: Acts as the master orchestration layer wrapping all underlying standalone scripts.

6. Complete Directory & File Structure

6.1 Annotated Repository Tree

The repository maintains a clean, modular structure separating source code, test data, models, and generated outputs:

```

sih_26154_project/ ■■■■ data/ ■■■■ sample_advisory.pdf # Primary test PDF (CERT-In Vulnerability
Note) ■■■■ test_text.txt # Raw text benchmark advisory ■■■■ models/ ■■■■ bart_summarizer/ # Local
cached weights for facebook/bart-large-cnn ■■■■ marian_translation/ # Local cached weights for
opus-mt-en-hi ■■■■ output/ ■■■■ extracted_fields.json # Generated structured JSON entities ■■■■
summary.txt # Generated English & Hindi summaries ■■■■ audio_briefing.wav # Synthesized offline audio
briefing ■■■■ src/ ■■■■ __init__.py # Package initialization marker ■■■■ ingest_pdf.py # PDF
parsing & layout extraction script ■■■■ extract_fields.py # Regex & NLP entity extraction module ■■■■
summarize.py # Hugging Face BART summarization script ■■■■ translate_hindi.py # MarianMT
English-to-Hindi translation script ■■■■ generate_audio.py # Text-to-Speech audio synthesis script ■■■■
app.py # Master Streamlit dashboard application ■■■■ requirements.txt # Fixed python dependency
manifest ■■■■ .gitignore # Git exclusion rules for air-gapped security ■■■■ README.md # High-level
developer documentation

```

7. Installation, Setup & Execution Guide

7.1 Hardware & Environment Prerequisites

Operating System: Linux (Ubuntu 20.04/22.04 LTS recommended), macOS, or Windows 10/11.

Python Version: Python 3.10.x or 3.11.x (3.10 recommended for maximum PyTorch compatibility).

System Hardware: Minimum 8 GB RAM (16 GB recommended); 10 GB free disk space for model weights.

System Software: ffmpeg (required for audio processing in some environments).

7.2 Step-by-Step Installation Commands

Follow these precise shell commands to setup an air-gapped local development environment:

```

# 1. Clone or extract project repository cd sih_26154_project # 2. Create isolated Python virtual
environment python3 -m venv venv # 3. Activate virtual environment # On Linux/macOS: source
venv/bin/activate # On Windows PowerShell: # .\venv\Scripts\Activate.ps1 # 4. Upgrade core package
managers pip install --upgrade pip setuptools wheel # 5. Install exact dependency package manifest pip
install -r requirements.txt

```

7.3 Execution Procedures & Verification Workflows

Option A: Running Individual Modules (CLI Verification)

To test individual pipeline stages independently, execute scripts directly from the project root:

```

# Test PDF Extraction python src/ingest_pdf.py --input data/sample_advisory.pdf # Test Field Extraction
python src/extract_fields.py --input data/sample_advisory.pdf --output output/extracted_fields.json #
Test Summarization python src/summarize.py --input data/sample_advisory.pdf # Test Audio Generation
python src/generate_audio.py --text "Critical vulnerability found in VMware." --output
output/test_audio.wav

```

Option B: Launching Full Web Dashboard

To launch the interactive dashboard, execute:

```
streamlit run app.py
```

8. End-to-End Execution Data Flow

8.1 Multi-Stage Pipeline Execution Walkthrough

When a security officer uploads a document via app.py, the system executes a deterministic 6-stage transformation pipeline:

- Step 1 (Ingestion):** app.py receives uploaded binary stream and saves it temporarily in data/.
- Step 2 (Text Cleaning):** ingest_pdf.py opens file via PyMuPDF, strips formatting noise, normalizes line breaks, and returns clean text string.
- Step 3 (Extraction):** extract_fields.py executes regular expressions over clean text, extracting CVE IDs, Severity, and Software, writing output to output/extracted_fields.json.
- Step 4 (Summarization):** summarize.py passes clean text to BART transformer, generating structured executive summary paragraphs.
- Step 5 (Translation):** translate_hindi.py translates the summary string into Devanagari Hindi text using local MarianMT.
- Step 6 (Synthesis):** generate_audio.py synthesizes English and Hindi summary strings into audio files saved in output/ and mounts audio controls on dashboard.

9. Testing, Validation & Experimental Results

9.1 Real-World Validation Strategy (CERT-In Corpus)

Testing was conducted using official CERT-In vulnerability advisories. CERT-In documents were selected because they represent authentic Indian government threat formats with complex mixed layout tables and standardized technical headers.

Pipeline Task	Input Test Sample	Expected Output	Actual Experimental Result	Status
PDF Parsing	CIVN-2024-0312 PDF	Full text extracted with zero missing sections.	100% text recovery. Line breaks cleanly normalized.	PASSED
CVE Extraction	Text containing 4 CVEs	JSON array containing all 4 CVE strings.	4/4 CVEs extracted correctly into JSON format.	PASSED
Severity Extraction	Header 'Severity: High'	Extract 'High'	Successfully isolated 'High' severity field.	PASSED
Summarization	1,200 word advisory	3-4 sentence English abstract.	Generated concise 85-word summary capturing core flaw.	PASSED
Hindi Translation	English summary string	Accurate Hindi Devanagari translation.	Readable Hindi text generated with minor technical term literalness.	PASSED

9.2 Identified Limitations in Current Prototypes

- 1. Regex Rule Scope:** Extraction depends on regular expressions. Unconventional text layouts or non-standard advisory formats may cause missing fields.
- 2. CPU Inference Latency:** Running BART-large summarization on CPU takes 8-15 seconds per document. GPU acceleration (CUDA) dramatically reduces this to <1.5 seconds.
- 3. Scanned PDF Limitation:** Documents that are image-only scans require an OCR pipeline (Tesseract OCR), which is planned for future iterations.

10. Security, Privacy & Air-Gapped Deployment

10.1 Air-Gapped Threat Model & Zero External API Policy

Operating in national technical intelligence contexts (NTRO) imposes absolute data containment mandates. Commercial LLM solutions expose organizations to telemetry logging, unauthorized model training on proprietary data, and interceptable network traffic.

Our Core Security Commitments:

- **Zero Outbound Traffic:** All models run strictly in local host memory.
- **Local Weight Storage:** Hugging Face weights are pre-downloaded and stored in models/.
- **Ephemeral File Execution:** Temp files generated during processing are cleaned automatically.

10.2 Sensitive Data Isolation & Repository Hygiene (.gitignore)

To prevent accidental commits of confidential advisories or heavy binary weights to version control, strict git rules are enforced:

```
# .gitignore rules for SIH 26154 venv/ __pycache__/ *.pyc data/*.pdf !data/sample_advisory.pdf output/*
!output/.gitkeep models/* .DS_Store *.log
```

11. SIH Requirements Alignment Matrix

SIH Problem Expectation	Our System Implementation	Verification Component / Evidence
Automated Document Ingestion	PyMuPDF parser handles PDF and text files cleanly.	src/ingest_pdf.py + Upload widget in app.py
Structured Information Extraction	Regex and NLP extract CVEs, Severity, Software, Mitigation.	src/extract_fields.py + output/extracted_fields.json
Executive Summarization	FB BART-Large-CNN generates abstractive English summary.	src/summarize.py + Executive Summary Box in UI
Regional Translation	MarianMT translates English summaries to Devanagari Hindi.	src/translate_hindi.py + Hindi View in UI
Audio Briefings (TTS)	Offline TTS synthesizes summaries into spoken audio files.	src/generate_audio.py + Streamlit Audio Player
100% Secure Local Deployment	Fully offline local architecture with zero cloud dependencies.	Local host running verification with network disconnected.

12. Project Status & Roadmap

12.1 Status Matrix of Platform Capabilities

Feature / Module	Implementation Status	Current Reality / Technical Notes
PDF & Text Ingestion	COMPLETED	Fully functional via PyMuPDF in ingest_pdf.py.
Regex Entity Extraction	COMPLETED	Extracts CVEs, CVSS, and Software into JSON.
BART Summarization	COMPLETED	Generates concise English abstractive summaries.
MarianMT Hindi Translation	COMPLETED	Translates summaries into Hindi Devanagari text.
TTS Audio Synthesis	COMPLETED	Renders offline audio files (.wav/.mp3).
Streamlit Unified Dashboard	COMPLETED	Interactive local web GUI operational.
SpaCy Fine-Tuned Named Entity Recognition	IN PROGRESS	Transitioning from pure Regex to hybrid SpaCy NER model.
Tesseract OCR for Scanned PDFs	PLANNED	Planned for Stage 2 hackathon refinement.
Multi-Language Support (Tamil/Telugu/Bengali)	PLANNED	Expanding IndicTrans2 coverage across additional regional languages.

13. Onboarding & Oral Defense Guide

13.1 Q&A; Master Sheet for Team Members

Use this master question-and-answer reference during SIH evaluation panels or internal technical onboarding:

Q1: What exact problem are you solving?

A: We automate the manual, time-consuming process of digesting unstructured cybersecurity advisories for organizations like NTRO, transforming dense PDFs into structured JSON data, executive summaries, regional translations, and audio briefs automatically.

Q2: What is the input to your system?

A: Unstructured documents including multi-page PDF vulnerability advisories (such as CERT-In notes), raw text security logs, and incident reports.

Q3: What happens internally during execution?

A: The system ingests the document, extracts raw text via PyMuPDF, parses technical metadata using Regex/NLP into JSON, summarizes text using a local BART transformer, translates summaries to Hindi via MarianMT, and synthesizes speech audio via offline TTS.

Q4: Which AI models are used and why?

A: We use **FB Bart-Large-CNN** for abstractive summarization and **Helsinki-NLP MarianMT** for Hindi translation. We chose them because they are highly accurate open-source transformers that run efficiently on local hardware without cloud dependencies.

Q5: What are the outputs produced by the platform?

A: Four primary deliverables: (1) Structured JSON metadata (CVEs, Severity, Software), (2) English Executive Summary, (3) Hindi Translated Summary, and (4) Playable Offline Audio Briefings.

Q6: Why is this relevant to NTRO?

A: NTRO processes sensitive national intelligence that cannot leave local premises. Our 100% local, air-gapped platform ensures absolute data sovereignty while boosting analyst efficiency.

Q7: What makes your approach secure?

A: Zero external API calls, offline local model execution, strict file hygiene via .gitignore, and zero data transmission over public networks.

Q8: What are the current limitations of your prototype?

A: Non-searchable scanned PDFs require an added OCR step, entity extraction relies partly on regular expressions, and CPU summarization takes several seconds per document without CUDA GPU acceleration.